Flask Framework in Python (Detailed Guide)

What is Flask?

Flask is a **lightweight (micro) web framework** written in Python. It is used to build:

- Websites
- Web Applications
- REST APIs
- Machine Learning Web Apps
- Admin Panels
- Dashboards
- Authentication Systems

Flask is called a **micro framework** because it provides only the essential features needed to build web applications. You can add additional functionality using extensions whenever you need it.

Why Learn Flask?

Flask is one of the most popular Python frameworks because it is:

- Easy to learn
- Lightweight
- Flexible
- Beginner friendly
- Highly customizable
- Good for small and large projects

Real World Companies Using Flask

Many companies have used Flask for parts of their applications, including:

- Instagram (certain services)
- Netflix
- Reddit
- Airbnb
- Lyft
- Pinterest

What Can You Build Using Flask?

You can create:

- Personal Portfolio Website
- Blog Website
- E-commerce Website
- Student Management System
- Employee Management System
- Hospital Management System
- Car Rental Website
- Book Selling Website
- Chat Application
- Banking Dashboard
- Machine Learning Prediction Website
- REST API

Flask Installation

Step 1: Create Virtual Environment

```
python -m venv myenv
```

Activate Environment

Windows

```
myenv\Scripts\activate
```

Linux / Mac

```
source myenv/bin/activate
```

Step 2: Install Flask

```
pip install flask
```

Check Installation

```
pip show flask
```

Version

```
flask --version
```

or

```
import flask
print(flask.__version__)
```

Your First Flask Program

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Welcome to Flask"

if __name__ == "__main__":
    app.run(debug=True)
```

Output

Open browser

```
http://127.0.0.1:5000/
```

Output

```
Welcome to Flask
```

Understanding Every Line

Import Flask

```
from flask import Flask
```

Imports Flask class.

Create Flask Application

```
app = Flask(__name__)
```

Creates the application object.

Think of it as the **main application**.

What is `__name__`?

`__name__` tells Flask where your application is located.

```
app = Flask(__name__)
```

If the file is executed directly:

```
__name__ = "__main__"
```

If imported:

```
__name__ = filename
```

Flask uses this information to find:

- Templates
- Static files
- Resources

Route

```
@app.route("/")
```

This is called a **Route**.

It connects a URL to a Python function.

```
URL
 ↓
Function
 ↓
Response
```

Example

```
@app.route("/")
def home():
    return "Home Page"
```

When user visits

```
localhost:5000/
```

home() runs.

Another route

```
@app.route("/about")
def about():
    return "About Page"
```

Visit

```
localhost:5000/about
```

Output

```
About Page
```

Multiple Routes

```
@app.route("/")
def home():
    return "Home"

@app.route("/contact")
def contact():
    return "Contact"

@app.route("/services")
def services():
    return "Services"
```

Run Server

```
app.run()
```

Starts development server.

Default

```
localhost:5000
```

Debug Mode

```
app.run(debug=True)
```

Advantages

- Auto restart
- Error page
- No need to restart every time

Folder Structure

```
Project/

│ app.py
│ requirements.txt
│

├── templates/
│       index.html
│       about.html
│
├── static/
│       css/
│       js/
│       images/
│
└── database/
```

Returning HTML

Instead of text

```
return "<h1>Hello</h1>"
```

Better

```
from flask import render_template

@app.route("/")
def home():
    return render_template("index.html")
```

---

templates/index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>

<h1>Welcome Flask</h1>

</body>
</html>
```

Static Folder

Contains

```
CSS

JavaScript

Images

Fonts
```

Example

```
static/

    css/style.css

    js/script.js

    images/logo.png
```

Use CSS

```
<link rel="stylesheet"
href="{{ url_for('static', filename='css/style.css') }}">
```

Image

```
<img src="{{ url_for('static', filename='images/logo.png') }}">
```

Dynamic Routing

```
@app.route("/user/<name>")
def user(name):
    return f"Welcome {name}"
```

Visit

```
/user/Krish
```

Output

```
Welcome Krish
```

Integer Route

```
@app.route("/age/<int:age>")
def age(age):
    return f"Age = {age}"
```

Visit

```
/age/21
```

Float

```
@app.route("/price/<float:value>")
```

Path

```
@app.route("/files/<path:file>")
```

URL Parameters (Query String)

```
localhost:5000/search?name=Krish
from flask import request

@app.route("/search")
def search():
    name = request.args.get("name")
    return name
```

HTTP Methods

Flask supports:

- GET
- POST
- PUT
- DELETE
- PATCH

GET

Used to fetch data.

```
@app.route("/students")
def students():
    return "Student List"
```

POST

Used to submit data.

```
@app.route("/login", methods=["POST"])
def login():
    return "Logged In"
```

Multiple Methods

```
@app.route("/login", methods=["GET","POST"])
def login():
    pass
```

HTML Form Example

HTML

```
<form method="POST">

<input type="text" name="username">

<input type="submit">

</form>
```

Python

```
from flask import request

@app.route("/", methods=["GET","POST"])
def home():

    if request.method=="POST":

        username=request.form["username"]

        return username

    return render_template("index.html")
```

Redirect

```
from flask import redirect
return redirect("/")
```

---

url_for()

Instead of

```
redirect("/home")
```

Use

```
redirect(url_for("home"))
```

Safer because it uses the function name rather than a hard-coded URL.

Templates (Jinja2)

Flask uses the **Jinja2** template engine.

Example

```
<h1>{{ name }}</h1>
```

Python

```
@app.route("/")
def home():

    return render_template(
        "index.html",
        name="Krish"
    )
```

Output

```
Krish
```

Loop

```
<ul>

{% for i in students %}

<li>{{ i }}</li>

{% endfor %}

</ul>
```

Python

```
students=["Ram","Shyam","Krish"]
```

If

```
{% if age>=18 %}

Adult

{% else %}

Minor

{% endif %}
```

Request Object

```
request.method

request.form

request.args

request.files

request.cookies

request.headers
```

Response Object

```
from flask import make_response

@app.route("/")
def home():

    response = make_response("Hello")

    return response
```

JSON Response

```
from flask import jsonify

@app.route("/api")
def api():

    return jsonify({

        "name":"Krish",

        "age":21

    })
```

Output

```
{
    "name":"Krish",
    "age":21
}
```

Sessions

Used to store user data temporarily.

```
from flask import session

app.secret_key="abc123"

@app.route("/login")
def login():

    session["user"]="Krish"

    return "Login Successful"
```

Read

```
session["user"]
```

Delete

```
session.pop("user")
```

Cookies

```
response.set_cookie("username","Krish")
```

Read

```
request.cookies.get("username")
```

File Upload

HTML

```
<form method="POST" enctype="multipart/form-data">

<input type="file" name="photo">

</form>
```

Python

```
file=request.files["photo"]

file.save(file.filename)
```

Error Handling

```
@app.errorhandler(404)
def page_not_found(error):
    return "Page Not Found",404
```

Blueprints

Blueprints split large applications into modules.

Example structure

```
Project/

app.py

auth/

product/

admin/
```

Database with Flask

Popular databases:

- SQLite
- MySQL
- PostgreSQL
- MongoDB

Example SQLite

```
import sqlite3

connection=sqlite3.connect("student.db")
```

Using **Flask-SQLAlchemy**:

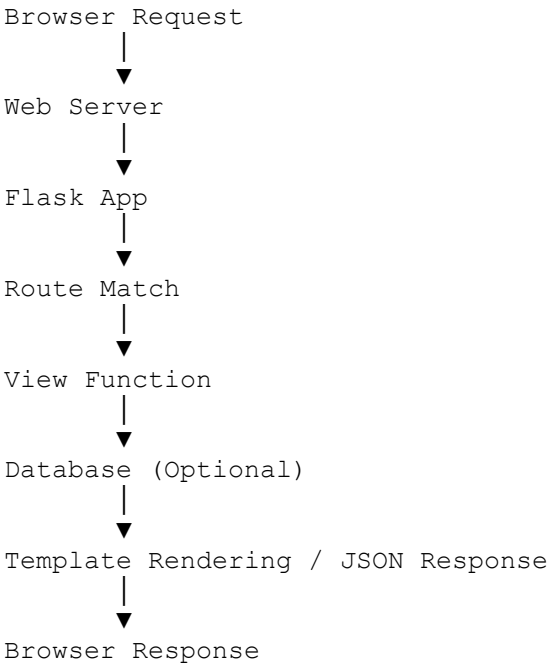
```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy(app)
```

Flask Extensions

Some commonly used extensions:

Extension	Purpose
Flask-SQLAlchemy	Database ORM
Flask-Login	Authentication
Flask-WTF	Forms & CSRF protection
Flask-Migrate	Database migration
Flask-Mail	Email
Flask-CORS	Cross-Origin Resource Sharing
Flask-JWT-Extended	JWT Authentication
Flask-RESTful	Build REST APIs

Flask Application Lifecycle



Flask Project Example (Student Management)

```
StudentManagement/

│ app.py
│ config.py
│ requirements.txt
│
├── templates/
│    ├── index.html
│    ├── students.html
│    ├── add_student.html
│    └── edit_student.html
```

```
│
├── static/
│   ├── css/
│   ├── js/
│   └── images/
│
├── models/
│   └── student.py
│
├── routes/
│   └── student_routes.py
│
└── database/
    └── student.db
```

Advantages of Flask

- Lightweight
- Easy to learn
- Flexible architecture
- Large extension ecosystem
- Excellent for REST APIs
- Ideal for beginners
- Easy to integrate with databases
- Good documentation

Disadvantages of Flask

- Fewer built-in features than full-stack frameworks
- Large projects require more architectural planning
- Many features depend on third-party extensions
- Not as opinionated as frameworks like Django

Flask vs Django

Feature	Flask	Django
Type	Micro Framework	Full-Stack Framework
Learning Curve	Easy	Moderate
Built-in Admin	❌	✅
ORM Included	❌ (extension)	✅
Authentication	Extension	Built-in
Flexibility	Very High	Moderate

Feature	Flask	Django
Best For	APIs, Small/Medium Apps	Large Applications

Interview Questions

1. What is Flask?
2. Why is Flask called a micro framework?
3. What is `Flask(__name__)`?
4. What is a route?
5. What is `@app.route()`?
6. What is the difference between GET and POST?
7. What is `render_template()`?
8. What is Jinja2?
9. What is `url_for()`?
10. What is the `request` object?
11. What is `jsonify()`?
12. What are sessions and cookies?
13. What are Blueprints?
14. What is the purpose of `debug=True`?
15. How do you connect a database in Flask?

Link : CHATGPT

Jinja2, Dynamic URL, and HTTP Methods in Flask (Detailed Guide)

These three topics are among the most important concepts in Flask.

---

1. Jinja2 Template Engine

What is Jinja2?

Jinja2 is the default **template engine** used by Flask.

It allows you to combine **HTML + Python data** to create dynamic web pages.

Without Jinja2:

```
<h1>Welcome Krish</h1>
```

This is **static HTML**.

With Jinja2:

```
<h1>Welcome {{ name }}</h1>
```

Python provides the value dynamically.

How Jinja2 Works

```
Python
   │
   │ send data
   ▼
Jinja2 Template
   │
   │ generate HTML
   ▼
Browser
```

Example

```
Python
```

```
name = "Krish"

↓

HTML

<h1>{{ name }}</h1>

↓

Browser

Welcome Krish
```

Folder Structure

```
Project/

│ app.py
│
├── templates/
│       index.html
│       about.html
│
└── static/
```

Important:

Flask automatically looks for HTML files inside the **templates** folder.

Example 1 : Display Variable

app.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():

    return render_template(
        "index.html",
        name="Krish"
    )

if __name__=="__main__":
    app.run(debug=True)
```

---

templates/index.html

```
<!DOCTYPE html>

<html>

<body>

<h1>Hello {{ name }}</h1>

</body>

</html>
```

Output

```
Hello Krish
```

Passing Multiple Variables

```
@app.route("/")
def home():

    return render_template(
        "index.html",
        name="Krish",
        age=21,
        city="Rajkot"
    )
```

HTML

```
<h2>Name : {{ name }}</h2>

<h2>Age : {{ age }}</h2>

<h2>City : {{ city }}</h2>
```

Output

```
Name : Krish

Age : 21

City : Rajkot
```

Jinja2 Expressions

```
{{ }}
```

Used to print data.

Example

```
{{ name }}
{{ age }}
{{ salary }}
```

Arithmetic

```
{{ 10+20 }}
```

Output

```
30
```

Multiply

```
{{ 10*5 }}
```

Output

```
50
```

Comments

```
{# This is comment #}
```

Browser will never display it.

If Statement

Python

```
@app.route("/")
def home():

    return render_template(
        "index.html",
        age=22
    )
```

HTML

```
{% if age >=18 %}

Adult

{% else %}
```

```
Minor

{% endif %}
```

Output

```
Adult
```

---

elif

```
{% if marks>=90 %}

A Grade

{% elif marks>=70 %}

B Grade

{% else %}

Fail

{% endif %}
```

For Loop

Python

```
students=[
"Krish",
"Rahul",
"Amit",
"Riya"
]
```

HTML

```
<ul>

{% for student in students %}

<li>{{ student }}</li>

{% endfor %}

</ul>
```

Output

- Krish
- Rahul
- Amit

```
• Riya
```

Loop Index

```
{% for student in students %}

{{ loop.index }}

{{ student }}

{% endfor %}
```

Output

```
1 Krish

2 Rahul

3 Amit
```

Dictionary Example

Python

```
student={
"name":"Krish",
"age":21,
"city":"Rajkot"
}
```

HTML

```
{{ student.name }}

{{ student.age }}

{{ student.city }}
```

List of Dictionary

Python

```
students=[

{"name":"Krish","age":21},

{"name":"Rahul","age":22}

]
```

HTML

```
<table border="1">

<tr>

<th>Name</th>

<th>Age</th>

</tr>

{% for s in students %}

<tr>

<td>{{ s.name }}</td>

<td>{{ s.age }}</td>

</tr>

{% endfor %}

</table>
```

Include Template

Header

```
header.html
```

Footer

```
footer.html
```

Main Page

```
{% include "header.html" %}

<h1>Home Page</h1>

{% include "footer.html" %}
```

Template Inheritance

Base Template

```
base.html
<!DOCTYPE html>

<html>
```

```
<body>

<header>

Website Header

</header>

{% block content %}

{% endblock %}

<footer>

Footer

</footer>

</body>

</html>
```

Child Page

```
{% extends "base.html" %}

{% block content %}

<h1>Home</h1>

{% endblock %}
```

This avoids repeating header and footer on every page.

Filters

Uppercase

```
{{ name|upper }}
```

Lowercase

```
{{ name|lower }}
```

Title

```
{{ name|title }}
```

Length

```
{{ students|length }}
```

---

url_for()

Instead of writing

```
<a href="/about">
```

Use

```
<a href="{{ url_for('about') }}">
```

Advantages

- No hardcoded URL
- Easy maintenance
- More secure
- Recommended by Flask

---

2. Dynamic URL

What is Dynamic URL?

A **dynamic URL** accepts values directly from the URL and passes them to a Flask function.

Instead of:

```
/user
```

We can use:

```
/user/Krish
```

Here, **Krish** is dynamic.

Basic Example

```
@app.route("/user/<name>")
def user(name):

    return f"Hello {name}"
```

Visit

```
http://127.0.0.1:5000/user/Krish
```

Output

```
Hello Krish
```

Integer Parameter

```
@app.route("/age/<int:age>")
def age(age):

    return f"Age = {age}"
```

Visit

```
/age/21
```

Output

```
Age = 21
```

If you visit:

```
/age/abc
```

Output

```
404 Not Found
```

Because `abc` is not an integer.

Float Parameter

```
@app.route("/price/<float:price>")
def price(price):

    return str(price)
```

URL

```
/price/150.75
```

Output

```
150.75
```

Path Converter

Useful for file paths.

```
@app.route("/file/<path:filename>")
def file(filename):

    return filename
```

Visit

```
/file/images/logo.png
```

Output

```
images/logo.png
```

Multiple Parameters

```
@app.route("/student/<name>/<int:age>")
def student(name, age):

    return f"{name} {age}"
```

URL

```
/student/Krish/21
```

Output

```
Krish 21
```

Build Dynamic Links

Instead of:

```
<a href="/user/Krish">
```

Use:

```
<a href="{{ url_for('user', name='Krish') }}">
```

Query String

Dynamic URLs can also use query parameters.

URL

```
/search?name=Krish
```

Python

```
from flask import request

@app.route("/search")
def search():

    name = request.args.get("name")

    return name
```

Output

```
Krish
```

Multiple Parameters

```
/search?name=Krish&age=21
name = request.args.get("name")
age = request.args.get("age")
```

---

3. HTTP Methods

What is HTTP?

HTTP (HyperText Transfer Protocol) is the protocol used for communication between the browser (client) and the web server.

```
Browser
    |
HTTP Request
    |
    ▼
Flask Server
    |
HTTP Response
    ▼
Browser
```

HTTP Request and Response

Example:

Browser requests:

```
GET /
```

Flask responds:

```
<h1>Welcome</h1>
```

Main HTTP Methods

Method	Purpose
GET	Read data
POST	Send/Create data
PUT	Update entire resource
PATCH	Update part of a resource
DELETE	Delete data

GET Method

Used to retrieve information.

```
@app.route("/")
def home():

    return "Home Page"
```

Browser sends:

```
GET /
```

Server returns:

```
Home Page
```

POST Method

Used to submit data.

HTML

```
<form method="POST">

<input name="username">

<input type="submit">

</form>
```

Python

```
from flask import request
```

```
@app.route("/", methods=["GET","POST"])
def home():

    if request.method=="POST":

        username=request.form["username"]

        return username

    return render_template("index.html")
```

PUT Method

Mostly used in REST APIs.

```
@app.route("/student", methods=["PUT"])
def update():

    return "Student Updated"
```

PATCH Method

Updates only specific fields.

Example

```
Old

Name
Age
City
```

PATCH

```
Only Age
```

DELETE Method

```
@app.route("/student", methods=["DELETE"])
def delete():

    return "Deleted"
```

---

request Object

Flask provides the `request` object to access incoming request data.

```
from flask import request
```

Common properties:

```
request.method       # GET, POST, PUT, DELETE
request.args         # Query parameters
request.form         # Form data
request.files        # Uploaded files
request.headers      # HTTP headers
request.cookies      # Cookies
request.json         # JSON body (if request is JSON)
```

GET vs POST

Feature	GET	POST
Purpose	Fetch data	Submit data
Data Location	URL (query string)	Request body
Visible in URL	✅ Yes	❌ No
Bookmarkable	✅ Yes	❌ No
Suitable for Passwords	❌ No	✅ Better (still use HTTPS)
File Upload	❌ No	✅ Yes
Typical Use	Search, view pages	Login, registration, forms

Real-World Examples

Website Feature	HTTP Method
Open home page	GET
Search for books	GET
Login form	POST
Register user	POST
Upload profile picture	POST
Update profile	PUT or PATCH
Delete account	DELETE

Summary

- **Jinja2** makes HTML dynamic by allowing Python variables, loops, conditions, filters, and template inheritance.

- **Dynamic URLs** capture values directly from the URL path (e.g., `/user/Krish`) or query string (e.g., `/search?name=Krish`).
- **HTTP methods** define the type of action the client wants to perform: `GET` (read), `POST` (create/submit), `PUT` (replace), `PATCH` (partial update), and `DELETE` (remove).

Mastering these three concepts is essential before moving on to Flask forms, authentication, databases (SQLAlchemy), REST APIs, and full-stack Flask projects.

Link : CHATGPT

Link : ALL